

PHP5.6・Zend Framework製サービスから学ぶ、 既存サービスの脆弱性の探し方

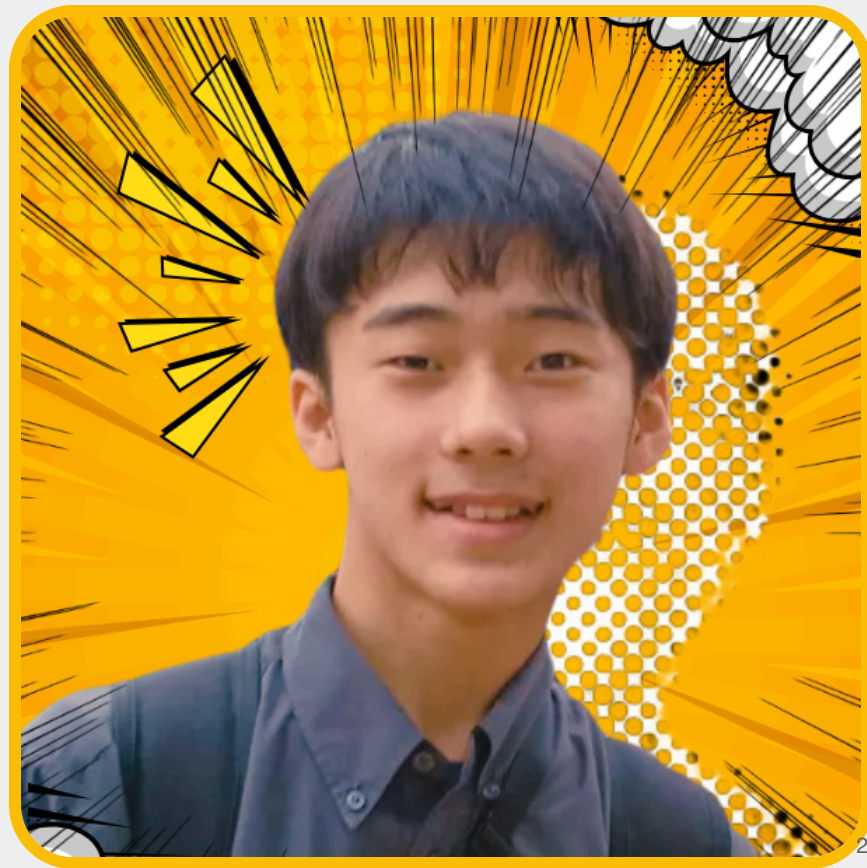
株式会社KOMPEITO 社内登壇

2026/08/06

田中博悠 / tanahiro2010 / サブカル性癖博士

自己紹介

- 田中博悠 / tanahiro2010
- サブカル性癖博士
- 三田学園高等学校 1年生
- システム開発グループ インターン
- GDG Greater Kwansai / Alpha+ Project
- 自認DevRel
- 読書 / バンジージャンプ / プログラミング



Security Researcher

っぽいこともしています

TL;DR

実際に見つけた脆弱性の種類

SQLi

入力がSQLに届く
までの経路を追う

RCE

任意コード実行に
つながる問題

認証バイパス

入口と権限のズレ
を見る

※ 公開資料にするため、実サービス名・再現手順・具体パラメータは扱いません

今日は攻撃手法ではなく、見つけ方に絞って話します

今、脆弱性って どう探していますか？

レビュー？ 診断ツール？ 勘？ ログ？ 怪しいコードを読む？

私はまず、3つを見ます



- 入口
Controller / API / 外部入力
- 境界
認証 / 権限 / DB・OSコマンド
- 歴史
古い実装 / コメント / 暫定対応

02. Controller継承を見る

MVCの入口はControllerに集まりやすいです



PHP5.6 + Zend Framework のようなMVC構成では、ここが探索の起点になります

継承元には「前提」が埋まりやすいです

AdminController

管理者であることを前提にした
処理

MemberController

ログイン済みユーザーを前提に
した処理

UserController

登録すれば通れる入口

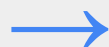
BaseController

認証なしでも動く可能性がある
入口

探すのは「前提のズレ」です

ゆるい入口

誰でも通れる
登録だけで通れる



重要な処理

決済 / 個人情報 / 管理操作

人を責める話ではなく、構造としてズレが起きやすい場所を見に行く話です

Controllerで見るチェックポイント

- API Controllerの継承元を一覧します
- 認証なし / ゆるい認証のControllerを見つけます
- その子クラスが重要処理に到達しないか確認します
- テスト用・暫定用の分岐が残っていないか見ます
- 「本番では無効」のつもりが、本当に無効か確認します

例: 開発用ショートカットやデバッグ分岐が、本番で有効になっていないか、など

03. スキャナで候補を増やす

php-vuln-scanner

未完成だけど、KOMPEITO向けに育てているOSSスキャナ

```
pipx run --spec https://github.com/tanahiro2010/php-vuln-scanner.git \  
php-vuln-scanner <target_path>
```

- 検出結果は「答え」ではなく「調査開始地点」です
- 過検知もあります
- 単一ファイルで見える範囲を中心に見ます
- PR歓迎です

出力をAIに渡すなら、丸投げしない

```
pipx run --spec https://github.com/tanahiro2010/php-vuln-scanner.git \
  php-vuln-scanner <target_path> > security.txt
```

- `security.txt` をそのまま渡すだけだと、ざっと見で終わりがちです
- 人間が「見る観点」を先に渡します
- AIには依存関係の確認・見落としチェックを手伝ってもらいます

古いファイルは「危険」 ではなく「歴史」です

長く動いているサービスには、コメント・命名・実装・
実際の利用状況のズレが蓄積します

コメント

「この関数は未使用」
でも現役で使われている

暫定対応

テスト用・一時対応の分岐が残る

認証前提

昔の入口を前提にした処理が残る

古い関数

今の設計と合わない処理が残る

Gitのログで優先度をつけます

```
git log --pretty=format: '%ad %h' --date=short -- path/to/file.php
```

- 更新が古いファイルをいくつか選びます
- 行数が多いファイルは、重要そうな関数から肉眼で見ます
- 依存関係が深いところはAIにも手伝ってもらいます
- 最後はUnitTestや再現手順で確認します

「古い = 危険」ではなく、「仕様と実装がズれているかもしれない」です

AIに渡すプロンプト例

<path> と、その依存関係を調査してください
このファイルは最終更新が古く、現在の認証・権限モデルと
ズレている可能性があります

外部入力が DB / ファイル / OSコマンド / eval 系処理に
到達する経路がないか確認してください

ファイル全部を雑に投げるより、観点を絞った方が検出率が上がります

デモ用リポジトリ

意図的に脆弱な学習用プロジェクトです
本番環境に置かないでください

```
git clone git@github.com:tanahiro2010/php-vuln.git
```

SQLi講義の資料・環境を使い回しているため、今回使わないファイルも含まれています

スキャナを実行します

```
pipx run --spec https://github.com/tanahiro2010/php-vuln-scanner.git \
  php-vuln-scanner ./php-vuln/sandbox/sqli-training-site-php
```

期待する流れ

1. いくつかの候補ファイルが出ます
2. ファイルパスを見に行きます
3. 外部入力がSQLへ届く経路を追います
4. SQL Injectionが成立する箇所を確認します

今日1時間だけやるなら

1. Controller継承を一覧します
2. 認証がゆるい入口から重要処理に行けないか見ます
3. 古いファイルを数個選びます
4. コメント・実装・現仕様のズレを見ます
5. スキャナやAIで候補を増やします
6. 最後はテストで確認します

Thanks for Listening!

聞いてくれてありがとう。
一緒にバンジー飛んでくれる人を探してます！